

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 09 -

SEGURANÇA NO CLUSTER

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	22
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	34

EMANIP

O QUE VEM POR AÍ?

Clusters Kubernetes modernos enfrentam grandes desafios de segurança, especialmente em ambientes com múltiplos serviços e equipes. Para garantir proteção, cada workload (aplicação rodando em um pod) deve possuir uma identidade própria, definida por ServiceAccounts, evitando o uso de credenciais genéricas. O controle de acesso é feito por meio do RBAC (Role-Based Access Control), que restringe as permissões de cada ServiceAccount ou usuário, seguindo o princípio do menor privilégio.

Boas práticas incluem: evitar permissões amplas (como curingas "*"), preferir roles limitadas ao namespace e nunca usar a ServiceAccount default inadvertidamente, pois ela possui permissões restritas. ServiceAccounts específicas para cada aplicação ajudam a delimitar claramente quem pode acessar o quê no API Server do Kubernetes.

Outro pilar essencial é garantir comunicações seguras via TLS, tanto dentro do cluster quanto externamente. Com o aumento dos microsserviços e requisitos de conformidade, criptografar o tráfego de rede se torna indispensável. Gerenciar certificados TLS manualmente é inviável em escala, pois erros podem causar certificados expirados ou uso inconsistente de criptografia.

É nesse contexto que o **cert-manager** se destaca: ele automatiza todo o ciclo de vida dos certificados no Kubernetes — emitindo, renovando e armazenando chaves de forma segura — e se integra com diversos provedores. Assim, a combinação de identidades seguras para workloads, controle estrito de ações (RBAC) e criptografia automatizada das comunicações (cert-manager) forma a base para uma segurança robusta em clusters Kubernetes hoje.

HANDS ON

Nesta seção prática, exploraremos exemplos de configuração de segurança em Kubernetes utilizando código em **Rust** com a biblioteca `kube-rs`. Os exemplos demonstram como criar `ServiceAccounts` e vinculá-las a pods, definir papéis de acesso mínimos via RBAC, integrar identidades de pods com provedores de nuvem (como **IRSA** no AWS e Workload Identity no GCP) e automatizar certificados TLS com recursos do **cert-manager**. Cada exemplo utiliza o client Rust para interagir com a API Kubernetes, ilustrando automação IaC (*Infrastructure as Code*) em nível de código. Assista à aula e depois leia sobre a implementação!

SAIBA MAIS

CRIANDO SERVICEACCOUNTS E ASSOCIANDO A PODS

Vamos começar criando uma ServiceAccount personalizada e associando-a a um pod. Em Rust, usamos o crate kube para acessar a API. O trecho abaixo cria uma ServiceAccount chamada analytics-sa no namespace analytics e lança um pod simples usando essa identidade:

```
use kube::{Client, Api};
use kube::api::{PostParams, ObjectMeta};
use k8s_openapi::api::core::v1::{ServiceAccount, Pod, PodSpec, Container};

#[tokio::main]
async fn main() -> Result<(), Box
```

Código-fonte 1 – Rust
Fonte: elaborado pelo autor (2025)

No código acima, após criarmos a ServiceAccount, configuramos o campo `service_account_name` do Pod para referenciá-la. Isso garante que o pod `test-pod` utilizará o token dessa ServiceAccount ao se comunicar com o API Server, em vez da ServiceAccount padrão do namespace. Na prática, isso significa que dentro do pod serão montadas credenciais (em `/var/run/secrets/kubernetes.io/serviceaccount`) correspondentes a `analytics-sa`.

Esse é o mecanismo nativo do Kubernetes para atribuir **identidade própria** a um pod. Conforme discutido, evitar o uso da ServiceAccount padrão é importante – aqui criamos uma identidade específica para o workload *analytics*.

DEFININDO ROLES E ROLEBINDINGS DE MENOR PRIVILÉGIO

Com a ServiceAccount criada, o próximo passo é conceder a ela somente as permissões necessárias para o seu funcionamento. Vamos supor que nossa aplicação `analytics` precisa ler ConfigMaps e listar Pods somente no namespace `analytics`. Podemos criar uma Role com essas permissões e um RoleBinding associando-a à ServiceAccount:

```
usek8s_openapi::api::rbac::v1::{Role, RoleRef, RoleBinding, PolicyRule, Subject};

letrole_api:Api<Role>=Api::namespaced(client.clone(), "analytics");
letread_role=Role {
  metadata:ObjectMeta { name:Some("analytics-read".into()), ..Default::default() },
  rules:Some(vec![
    PolicyRule {
      api_groups:Some(vec!["".into()]), // grupo core (v1)
      resources:Some(vec!["pods".into()]),
      verbs:vec!["get".into(), "list".into()],
      ..Default::default()
    },
    PolicyRule {
      api_groups:Some(vec!["".into()]),
      resources:Some(vec!["configmaps".into()]),
      verbs:vec!["get".into()],
      ..Default::default()
    }
  ]),
  ..Default::default()
};
role_api.create(&PostParams::default(), &read_role).await?;

letrb_api:Api<RoleBinding>=Api::namespaced(client.clone(), "analytics");
letbinding=RoleBinding {
  metadata:ObjectMeta { name:Some("bind-analytics-read".into()), ..Default::default() },
  role_ref:RoleRef {
    api_group:"rbac.authorization.k8s.io".into(),
    kind:"Role".into(),
    name:"analytics-read".into(),
  },
  subjects:Some(vec![
    Subject {
      kind:"ServiceAccount".into(),
      name:"analytics-sa".into(),
      namespace:Some("analytics".into()),
      ..Default::default()
    }
  ]),
  ..Default::default()
};
rb_api.create(&PostParams::default(), &binding).await?;
```

Código-fonte 2

Fonte: elaborado pelo autor (2025)

No Role acima, especificamos explicitamente as ações permitidas (verbs) e os recursos. Algumas boas práticas de RBAC: - **Escopo mínimo**: usamos uma Role namespaced (em vez de ClusterRole) já que as necessidades são restritas ao namespace *analytics*. Isso evita expor recursos de todo cluster desnecessariamente.

Concedemos somente leitura (*get*, *list*) e somente em **pods** e **configmaps**, ao invés de usar verbos como *create* ou recursos desnecessários. Cada PolicyRule é bem delimitado, não usamos "*" nem em recursos nem em verbs. Por exemplo, poderíamos listar todos os tipos de objeto com "*" ou todos verbs, mas isso quebraria

o princípio de mínimo privilégio, e o RoleBinding referência especificamente a ServiceAccount analytics-sa (note o campo subjects com kind: ServiceAccount e name/namespace). Isso vincula essa Role *apenas* àquela identidade.

Com isso, o pod test-pod rodando com ServiceAccount analytics-sa terá **exatamente** essas permissões e nenhuma outra. Se dentro do contêiner alguém tentar, por exemplo, deletar um Pod ou ler um Secret, a solicitação será negada pelo API Server. Assim, até mesmo se o contêiner for comprometido, o invasor fica limitado ao que definimos nesta Role – minimizando o impacto.

INTEGRAÇÃO DE SERVICEACCOUNTS COM PROVEDORES EXTERNOS

Além de controlar o acesso ao **cluster**, muitas aplicações executadas no Kubernetes precisam acessar serviços de nuvem (AWS, GCP, Azure) de forma segura, sem embutir chaves de acesso em código. Os provedores cloud oferecem mecanismos de **federar a identidade do pod (ServiceAccount) a uma identidade cloud**, permitindo emissão de credenciais temporárias.

Dois exemplos populares são: **IRSA (IAM Roles for Service Accounts)** no Amazon EKS e o **Workload Identity** no Google GKE (ou *Azure Workload Identity* no AKS). Em ambos os casos, a ideia central é: **associar uma ServiceAccount do Kubernetes a uma credencial gerenciada no provedor**, de forma que o pod possa obter tokens/cloud automaticamente.

No EKS com IRSA, o procedimento envolve estabelecer um provedor OIDC para o cluster e criar uma IAM Role no AWS com trust policy apontando para esse provedor. Depois, uma ServiceAccount é anotada com o ARN da Role desejada. Podemos fazer isso via kube-rs facilmente atualizando as anotações da ServiceAccount. Suponha que já temos uma IAM Role `arn:aws:iam::123456789012:role/S3Reader` que dá acesso de leitura a um bucket S3. O código abaixo adiciona a annotation requerida:


```
usekube::api::PatchParams;
usekube::api::Patch;

// Prepara um patch JSON Merge com a annotation da IRSA
let patch = serde_json::json!({
  "metadata": {
    "annotations": {
      "eks.amazonaws.com/role-arn": "arn:aws:iam::123456789012:role/S3Reader"
    }
  }
});
sa_api.patch("analytics-sa", &PatchParams::default(),
&Patch::Merge(&patch)).await?;
```

Código-fonte 3
Fonte: elaborado pelo autor (2025)

Após aplicar essa alteração, o Kubernetes **projeta** automaticamente tokens JWT nos pods vinculados à ServiceAccount que podem ser trocados por credenciais AWS de curto prazo. Assim, dentro do pod, se o AWS SDK estiver configurado, a aplicação passa a ter permissão nos serviços definidos pela Role AWS (neste exemplo, acesso de leitura ao S3).

Nenhuma chave permanente foi inserida no contêiner – a autenticação é feita através do token de ServiceAccount validado pelo provedor OIDC do cluster e trocado pela Role. Isso melhora muito a segurança, pois elimina o compartilhamento de segredos estáticos e **restringe o alcance da credencial AWS ao contexto do pod** específico. Importante: **cada aplicação deve ter sua própria IAM Role via IRSA**, evitando que múltiplos pods usem a mesma Role. É uma *best practice* isolar ao máximo (um pod/serviço = uma Role IAM), para delimitar falhas e facilitar auditoria.

De forma análoga, no GKE habilitamos Workload Identity para o cluster e associamos uma ServiceAccount Kubernetes a uma **Google Service Account (GSA)** do IAM do Google Cloud. Isso também é feito adicionando uma annotation (por exemplo: "iam.gke.io/gcp-service-account": "meu-servico@projeto.iam.gserviceaccount.com"). A lógica é parecida: o pod obtém um token JWT que o Google confere e emite um token de acesso GCP limitado às permissões daquela GSA.

A **identidade do workload** fica, assim, federada com uma identidade de cloud, sem a necessidade de armazenar secrets no pod . O Azure AKS historicamente usava o **AAD Pod Identity** (agora deprecado) e atualmente suporta *Workload Identity* integrado ao Azure Entra ID, de forma que a ServiceAccount se torna elegível a receber um token JWT que a Azure Entra troca por um token de acesso para uma Managed Identity do Azure. Em todos os casos, o padrão é semelhante: **tokens curtos e federados** em vez de chaves longas. Isso reduz superfície de ataque e acopla permissões estritamente àquela execução do pod.

TLS AUTOMATIZADO COM CERT-MANAGER (ISSUER, CERTIFICATE, ETC.)

Por fim, veremos como automatizar a gestão de certificados TLS dentro do cluster usando o **cert-manager** via kube-rs. O cert-manager introduz novas CRDs (Custom Resource Definitions) como **Issuer**, **ClusterIssuer** e **Certificate** que controlam a emissão de certificados. Podemos interagir com essas CRDs em Rust utilizando objetos dinâmicos ou definindo tipos customizados via kube-derive. Para simplificar, mostraremos o fluxo usando YAML – entendendo que podemos aplicar esses manifestos via código ou kubectl.

Considere o cenário: queremos configurar certificados internos autoassinados para comunicação mútua entre dois serviços (service-a e service-b), sem depender de AC externa. Podemos utilizar um Issuer do tipo SelfSigned para gerar uma CA raiz local e depois emitir certificados para cada serviço com um CA Issuer baseado na raiz:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned-root-issuer
spec:
  selfSigned: {} # Issuer autoassinado (gera sua própria CA)

---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: my-root-ca
spec:
  secretName: root-ca-secret
  isCA: true
  commonName: "MyOrg Root CA"
  issuerRef:
    name: selfsigned-root-issuer
    kind: Issuer

---
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: cluster-ca-issuer
spec:
  ca:
    secretName: root-ca-secret # referência à chave/ca gerada acima

---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: service-a-cert
spec:
  secretName: service-a-tls
  duration: 2160h # 90 dias
  renewBefore: 360h # renovar 15 dias antes
  commonName: "service-a.default.svc.cluster.local"
  dnsNames:
    - service-a
    - service-a.default.svc.cluster.local
  issuerRef:
    name: cluster-ca-issuer
    kind: Issuer
```

Código-fonte 4

Fonte: elaborado pelo autor (2025)

No exemplo, o primeiro Issuer (selfsigned-root-issuer) emite um certificado raiz autoassinado, que é armazenado no Secret root-ca-secret. Depois definimos um Issuer cluster-ca-issuer do tipo CA que usa essa raiz para assinar outros certificados. Por fim, criamos um Certificate para service-a (armazenado em service-a-tls secret) com CN e SANs correspondentes ao DNS interno do serviço no cluster.

O cert-manager ao ver esse objeto Certificate irá automaticamente gerar um CSR, assiná-lo com a CA especificada e preencher o Secret com tls.crt (certificado)

e `tls.key` (chave privada). O serviço `service-a` poderia montar esse `Secret` em volume e configurar seu servidor HTTP para usar o TLS. O campo `renewBefore` garante renovação antecipada (neste caso, 15 dias antes de expirar).

Tudo isso, poderia ser feito manualmente (usando `openssl`, gerando e distribuindo chaves), mas o **cert-manager automatiza integralmente, evitando expiração acidental de certificados e configurações díspares**. Com pequenos ajustes, poderíamos usar ACME para certificados públicos (Let's Encrypt) ou integrar com PKIs corporativas – bastaria trocar o Issuer para `ClusterIssuer` ACME ou Vault, etc. Em Rust, poderíamos criar esses objetos via `Api<DynamicObject>` apontando para o group `cert-manager.io`. Exemplo de criação dinâmica:

```
use kube::api::{Api, DynamicObject};

// CRD Issuer (DynamicObject porque a definição exata não está no
// k8s_openapi)
let issuers: Api<DynamicObject> = Api::namespaced(
    client.clone(), "default").with_kind(
        "Issuer"
    ).within_group(
        "cert-manager.io"
    ).within_version("v1");
```

Código-fonte 5

Fonte: elaborado pelo autor (2025)

Observação: o pseudo-código acima ilustra o conceito; *kube* também permite definir CRDs via macros. Uma alternativa seria definir `structsRust` correspondentes ao spec do Issuer e Certificate, usando `#[derive(CustomResource)]` do `kube-derive`.

O importante é entender que, por trás dos bastidores, o `cert-manager` mantém um controlador no cluster que detecta esses objetos e realiza as ações de acordo: geração de chaves, assinatura, criação de `Secrets` e agendamento de renovações. Para cada `Ingress` ou `Serviço` que precisar de TLS, podemos simplesmente criar um objeto `Certificate` e confiar que `cert-manager` gerenciará o resto. **A automação da PKI dentro do cluster provê um enorme alívio operacional, garantindo consistência na emissão e rotação de certificados TLS**, um aspecto vital de segurança (certificados expirados ou mal configurados podem derrubar sistemas inteiros).

CONTROLE DE ACESSO: RBAC VS. ABAC MODELADOS COMO GRAFOS

Role-Based Access Control (RBAC) e **Attribute-Based Access Control (ABAC)** são paradigmas distintos para autorização. No RBAC, permissões são concedidas com base em **funções (roles)** atribuídas a sujeitos; no ABAC, decisões são tomadas avaliando **atributos** dos sujeitos, recursos, contexto etc. Podemos modelar o RBAC classicamente com três conjuntos fundamentais: U (usuários, incluindo contas de serviço), R (roles) e P (permissões, i.e. ações em recursos). Há então relações de associação: $UA \subseteq U \times R$ (atribuição usuário->role) e $PA \subseteq R \times P$ (atribuição role->permissão).

Essa estrutura pode ser vista como um **grafo bipartido duplo** – de um lado usuários ligados a roles, e roles ligados a permissões. A efetiva permissão de um usuário flui através desses vínculos: um usuário u tem uma permissão p se $\exists$ role r tal que $(u,r) \in UA$ e $(r,p) \in PA$. Análises de segurança podem então ser reduzidas a procurar caminhos nesse grafo – por exemplo, **verificar fluxo de permissões** equivale a checar se um determinado usuário alcança um nó de permissão proibido via alguma sequência de associações.

No Kubernetes, o RBAC implementa esse modelo com algumas particularidades: "Subjects" podem ser usuários, grupos ou ServiceAccounts; "Resources" abrangem objetos do API (pods, deployments, etc.) muitas vezes com granularidade de verbo; e "Roles/ClusterRoles" fazem o papel das roles no grafo.

A autorização se resume a: $Subject \rightarrow RoleBinding \rightarrow Role \rightarrow Permissão$. Essa linearidade (sujeito vinculado a role, role listando ações permitidas) torna a avaliação do acesso relativamente simples (basta conferir se existe um binding dando aquela permissão).

Entretanto, surgem questões mais complexas como por exemplo: *É possível que um usuário adquira determinado privilégio através de uma combinação de roles?*

Em RBAC puro (estático), isso se responde caminhando no grafo de atribuição. Em modelos RBAC administrativos (onde há regras para quem pode atribuir roles a quem), a análise torna-se mais difícil – na verdade, foi provado que decidir se um

usuário **poderá vir a obter** uma permissão (chamado problema de *safety*) é, no caso geral, um problema co-NP-completo (essencialmente NP-difícil).

Intuitivamente, isso acontece porque se roles podem conceder outras roles (hierarquias complexas, previsões futuras), a verificação vira uma busca potencialmente exponencial. Por sorte, o Kubernetes não possui um modelo nativo de delegação de role (apenas admins centralizados definem RoleBindings), o que simplifica muito o cenário: o RBAC do Kubernetes é estático e explicitamente **aditivo** (não há negações, apenas somas de permissões). Assim, a **auditabilidade** é maior – podemos listar quem tem acesso a quê de forma determinística.

Já o **ABAC** define políticas baseadas em atributos e é muito mais difícil de modelar por grafos simples. Uma política ABAC é essencialmente uma função booleana $f(\text{sujeito}, \text{objeto}, \text{contexto})$ que decide allow/deny. Podemos imaginar cada atributo como um predicado, e a política como uma fórmula lógica sobre esses predicados. Por exemplo: *allow leitura se subject.department = objeto.department OU subject.cargo = "Gerente"* – uma regra dessas não se traduz facilmente em nós de grafo, pois envolve condições arbitrárias.

Em notação formal, o ABAC se assemelha a um conjunto de **regras de inferência**: um pedido de acesso é autorizado se satisfizer todas as premissas de alguma regra. Apesar de poderoso, ABAC puro costuma ser menos utilizado em Kubernetes. Havia suporte experimental via arquivo JSON estático de políticas, mas desde v1.6 o RBAC substituiu-o quase completamente e o ABAC no Kubernetes hoje é considerado obsoleto (deprecated).

Um dos motivos é a dificuldade de gerenciamento e compreensão das políticas ABAC, que podem se tornar complexas e opacas, enquanto RBAC é mais direto e fácil de verificar. Em termos de grafos, o ABAC poderia ser visto como um **hipergrafo** onde os nós são atributos e a política é uma hiper-aresta ligando múltiplas condições a um resultado.

A **análise de segurança de ABAC** recai na avaliação de condições lógico-condicionais; em casos gerais, reduzir um sistema ABAC a algo decidível ou fácil de inspecionar não é trivial. Por isso, Kubernetes e outras plataformas convergiram ao RBAC: **menos expressivo, porém previsível**. Um modelo formal de RBAC nos permite garantir propriedades de segurança (como separação de deveres, ausência

de caminhos de privilégio não intencionais) com relativa facilidade computacional, enquanto ABAC sacrifica simplicidade por flexibilidade.

FUNDAMENTOS DE PKI, CERTIFICADOS X.509 E CADEIA DE CONFIANÇA

A segurança de comunicações e autenticações em Kubernetes (e em sistemas distribuídos em geral) apoia-se fortemente em **infraestrutura de chaves públicas (PKI)**. Formalmente, uma PKI consiste em um conjunto de entidades que possuem pares de chaves criptográficas (pública/privada) e um conjunto de relações de confiança estruturadas hierarquicamente.

No topo da cadeia está a **Autoridade Certificadora Raiz (Root CA)**, que assina certificados de entidades intermediárias ou finais. Um **certificado X.509** é essencialmente um documento digital que vincula uma identidade (nome comum, DNS, endereço etc.) à sua chave pública, através de uma assinatura feita por uma CA confiável. A assinatura funciona matematicamente como:

$$Cert = \{ dados, chave pública, validade temporal, \dots \}_{assinado\ pela\ CA}$$

Para verificar a validade e de um certificado, o consumidor usa a chave pública da CA emissora para validar a assinatura digital presente no certificado. Essa estrutura é recursiva: a CA emissora pode ter seu próprio certificado assinado por uma CA superior, formando assim uma **cadeia de confiança**. No topo dessa cadeia, a Root CA é autoassinada (ela assina o próprio certificado). Assim, para confiar em qualquer certificado final, você deve confiar na Root CA correspondente (*trust anchor*).

No contexto do Kubernetes, a PKI é onipresente. O próprio cluster mantém uma CA raiz que assina os certificados do API Server, do kubelet, do etcd e assim por diante. Quando um cliente `kubectl` se conecta ao API Server, ele valida o certificado do API Server contra a CA do cluster (geralmente armazenada em arquivo local).

Internamente, o Kubernetes adota um modelo de múltiplas CAs para segregar contextos: por exemplo, uma CA própria para etcd (isolando o escopo das chaves etcd, já que se comprometida não compromete outras comunicações) e outra para front-proxy (usada em extensões de API). Isso implementa formalmente o princípio de **compartimentalização de confiança**: definimos conjuntos disjuntos de componentes

e damos a cada conjunto sua própria autoridade de certificação, de modo que a confiança não transite indevidamente entre domínios diferentes.

Em termos matemáticos, podemos afirmar que as CAs distintas definem diferentes *domínios de validade* para pares de chaves (K^{+pub}, K^+) . Dessa forma, provamos que, se um domínio for comprometido (ex: CA etcd exposta), ele não autoriza ações em outro domínio (não pode assinar um certificado válido para o API server, pois o API server só confia na CA raiz primária).

Um certificado X.509 possui campos importantes do ponto de vista formal: *Subject* (identidade do dono), *Issuer* (identidade da CA que assinou), *Validity* (*NotBefore/NotAfter*), *Public Key* (do *subject*), *Signature* (feita com privada da CA), e extensões como *Key Usage* e *Extended Key Usage* que especificam usos permitidos.

Em ambiente Kubernetes, vemos perfis específicos: certificados de servidor (API server, etcd) têm extensão `serverAuth`; certificados de cliente (entre componentes, como kubelet->APIserver) têm `clientAuth`. Essa formalização garante que chaves não sejam usadas indevidamente (um certificado com apenas `clientAuth` não deveria ser aceito para serviço de servidor TLS, por exemplo).

A **hierarquia de confiança** implica que, para verificar um certificado de pod ou usuário, é necessário construir a cadeia até uma CA raiz conhecida. O Kubernetes hoje não implementa mecanismos de revogação online (CRL ou OCSP) – o que significa, do ponto de vista formal, que um certificado é considerado válido até expirar, a menos que o administrador manualmente retire a confiança da CA emissora.

Por isso, uma boa prática de segurança é **usar certificados com vida curta** (por exemplo, tokens de ServiceAccount do tipo `bound` têm validade de minutos), reduzindo a janela de exploração caso uma chave vaze.

A matemática de chaves públicas assegura que, assumindo a dificuldade de fatorar números ou solver logaritmos discretos (dependendo do algoritmo usado, RSA ou ECC), a única forma de forjar a identidade de outrem é comprometendo sua chave privada ou quebrando a confiança de uma CA.

Por isso, na segurança de cluster, mantemos as chaves privadas das CAs guardadas a sete chaves (idealmente em HSMs ou KMS) e rotacionamos certificados periodicamente para limitar danos caso um vazamento ocorra. A **PKI bem projetada** do Kubernetes – com múltiplas autoridades e escopos – reforça a robustez: mesmo

que a chave de API server vaze, ela não dá acesso ao etcd, por exemplo, graças à separação de autoridades. Essa separação e o uso de certificados X.509 formam a espinha dorsal de autenticação mútua dentro do cluster, habilitando a confiança zero trust que mencionamos.

POLÍTICAS DE SEGURANÇA COMO CÓDIGO: OPA, LÓGICA DE PREDICADOS E ANÁLISE ESTÁTICA

Definir políticas de segurança em alto nível, muitas vezes *como código*, é uma tendência que tornar as configurações mais auditáveis e versionáveis. Um exemplo marcante é o **OPA (Open Policy Agent)** com sua linguagem declarativa **Rego**. Podemos entender o Rego como uma linguagem de **lógica de predicados de primeira ordem** com sintaxe influenciada por Datalog. Em Rego, escrevemos afirmações que devem ser verdadeiras para que algo seja permitido.

Por exemplo, poderíamos expressar em pseudo-lógica: `deny[msg]:- input.request.kind.kind = "Pod", not input.request.object.spec.containers[_].resources.limits, msg = "Missing limits"` (essa é exatamente a forma de uma regra Rego que nega criação de pod sem resources limit).

Cada regra em Rego é basicamente um predicado definido por conjunção de condições (e possivelmente com quantificadores implícitos em forma de iteração). A política final é o conjunto de todas essas regras; a autorização é *allow* se nenhuma regra *deny* aplicar, por exemplo.

Do ponto de vista formal, isso se aproxima de especificar um conjunto P de fórmulas lógicas. Avaliar uma política equivale a executar um **motor de inferência** sobre essas fórmulas dado um modelo (os atributos do request). O OPA faz isso de forma eficiente através de otimizações e indexações, mas teoricamente podemos considerar Rego Turing-completo (ele suporta recursão limitada, laços via iteração sobre dados, entre outras medidas).

Esse poder expressivo traz um **trade-off importante**: quanto mais expressiva a linguagem de políticas, mais difícil é provar propriedades sobre elas. Por exemplo, garantir *statically* que "nenhum pod privilegiado será admitido" é simples de verificar

se usamos apenas RBAC (basta checar que nenhuma Role permite criar pods com escalonamento de privilégios).

Mas se tivermos uma política OPA complexa que, digamos, depende de labels de ambiente e uma lista de exceções por namespace, **verificar automaticamente a consistência ou ausência de contradições pode ser indecidível em geral** (análoga ao problema da parada, já que políticas podem incorporar lógica arbitrária).

Em outras palavras, a **análise estática de políticas expressivas é computacionalmente custosa**. Ainda assim, para evitar surpresas, as práticas recomendadas incluem manter as políticas modulares e específicas, facilitando testes unitários (OPA fornece teste de políticas como se fossem funções, para várias entradas). Há também pesquisas que propõem restrições em linguagens de políticas para torná-las *decidíveis*.

Uma consideração é o balanço entre **expressividade vs auditabilidade**. RBAC é altamente auditável – você pode listar roles e bindings e facilmente entender quem pode fazer o quê. Porém, não expressa cenários condicionais (ex: "permitir ação X apenas em horário comercial" ou "negar se imagetag não for 'stable'").

Políticas definidas como OPA podem expressar isso (ex.: usar `time.now` ou validar campo de imagem), mas depois você precisa auditar código lógico. Empresas e projetos escolhem um ponto nesse espectro. Formalmente, preferir OPA/Rego é optar por maior poder de expressão, equiparável a ter *controle de acesso baseado em lógica de predicados*. Isso se relaciona a ABAC, mas OPA vai além: é **Policy as Code**, não só para autorização Kubernetes, mas para qualquer decisão (pode proteger APIs, CI/CD, infra cloud).

Ainda assim, conseguimos tirar valor prático dessas políticas fazendo **análise estática parcial** – por exemplo, checando um repositório de manifests contra as políticas (ferramentas como Conftest fazem isso) ou usando a própria OPA no modo *debug* para avaliar regras não satisfeitas. Isso não garante perfeição, mas pega *violações óbvias* antecipadamente.

Em termos de complexidade, a avaliação de uma política Rego específica com input finito é decidível (basta executá-la), e mecanismos de otimização (indexação de regras, ordenação de predicados) tornam isso rápido na maioria dos casos.

ROTAÇÃO DE SEGREDOS E CONSISTÊNCIA NA DISTRIBUIÇÃO DE CHAVES

Segredos – como senhas, tokens, chaves de criptografia – têm um ciclo de vida e precisam ser eventualmente rotacionados (trocados por novos) para reduzir o tempo de exposição caso vazem. A **rotação de segredos** é um processo delicado pois envolve emitir novas credenciais, distribuir aos consumidores e invalidar as antigas, tudo isso sem interromper sistemas que dependem delas.

Há duas estratégias: **rotação com período de convivência** ou rotação instantânea. No mundo real, a rotação instantânea é impraticável em sistemas distribuídos, pois não há lock global sincronizando todas as partes. Então adota-se *convivência*: por um tempo, ambos os segredos são aceitos, até termos confiança de que todos migraram.

Por exemplo, o HashiCorp Vault distingue **segredos estáticos com rotação** de **segredos dinâmicos**. Um segredo dinâmico é gerado sob demanda para cada cliente e geralmente tem TTL curto – isso remove a necessidade de rotacionar porque cada uso é efêmero (um token JWT de 15min, por exemplo).

Já os segredos tradicionais (estáticos) precisam de rotação. Um bom esquema de rotação (conforme descrito pela HashiCorp) envolve *quatro fases*:

- **Criação** de nova versão;
- **Teste/validação** da nova credencial;
- **Ativação** – começa-se a usar a nova em produção (e possivelmente emitir notificações para pods reconectarem);
- **Desativação/remoção** da antiga.

Durante a fase 3, muitas ferramentas permitem que **duas versões fiquem ativas simultaneamente** (a antiga e a nova). Isso garante que quem ainda não pegou o segredo novo não pare de funcionar imediatamente – um consumidor antigo ainda pode usar versão anterior até um deadline.

Essa sobreposição temporária deve ser curta o suficiente para evitar a manutenção indefinida de segredos antigos, mas **longa o bastante para permitir que todos os consumidores realizem a atualização com segurança**.

No Kubernetes, um exemplo disso é a rotação de certificados do kubelet: podemos configurar para que os certificados sejam renovados automaticamente e mantidos ambos até que a troca seja concluída sem afetar comunicação.

Outro aspecto é a **distribuição consistente de chaves**. O Kubernetes oferece recursos para ajudar nisso, como o *EncryptionConfiguration* para segredos no etcd. Podemos ter múltiplas chaves de criptografia configuradas, com uma marcada como **ativa** para novas escritas e as anteriores permanecendo para leitura retroativa.

Esse padrão é comum em sistemas com **múltiplas chaves KMS/DEK em camadas**, onde novas criptografam dados novos, mas as antigas não são descartadas até recriptografarmos tudo ou garantirmos que dados cifrados com elas já não existem mais. No contexto do etcd, a rotação de chave mestra envolve: adicionar a nova chave no config do API Server, fazer rollout nos nós de controle um a um (assegurando que todos conhecem ambas as chaves) – importante: todos nodes de controle devem ter config idêntica, senão um escreveria com chave que outro não sabe ler.

Após isso, pode-se recriptografar os Secrets para usar apenas a chave nova e então retirar a antiga do config. Note que durante o rollout, se *qualquer* API server não tiver a nova chave, ele falhará ao ler segredos escritos pelo que já rotacionou – daí a necessidade de **ordem e consistência** na distribuição da configuração de chave, sob pena de criar um split-brain de criptografia.

Em sistemas distribuídos, analisamos consistência com modelos semelhantes ao de transações em banco de dados ou algoritmos de consenso. A rotação segura de um segredo assemelha-se a um commit de transação distribuída: queremos garantir que **todos os participantes concordem na versão do segredo**.

Se mesmo um participante estiver defasado (ainda usando a credencial antiga enquanto o resto já rejeita), haverá falha de autenticação. Por isso, muitas soluções optam por rotação controlada centralmente e monitoramento: por exemplo, **Kubernetes Secrets podem ser monitorados por controladores** (como External Secrets Operators) que automaticamente atualizam aplicações quando o valor muda; *sidecars* de atualização podem sinalizar a aplicação a recarregar credenciais. Formalmente, podemos pensar em um *grafo de dependência* onde nós são componentes (pods, serviços) e todos que dependem do segredo S devem receber S'

antes do ponto de corte. Ferramentas de orquestração podem aplicar teoria de grafos (ordenamento topológico, rollback em caso de falha) para esta distribuição.

Do ponto de vista de segurança formal, a rotação periódica melhora a distribuição de valores de chave no tempo, reduzindo a *entropia temporal* de um atacante (menos tempo para crackear ou exfiltrar uma chave). Entretanto, se mal feita, pode introduzir indisponibilidade (que é outra faceta da segurança, a disponibilidade

Aplicando princípios de algoritmos distribuídos, conseguimos rotacionar segredos sem caos: **coordenar, notificar, validar e somente então invalidar** a versão anterior. Esse encadeamento cauteloso garante consistência mesmo em clusters com dezenas de microsserviços usando a mesma credencial.

MERCADO, CASES E TENDÊNCIAS

Nesta seção, examinamos como as grandes plataformas de Kubernetes gerenciados aplicam esses conceitos de segurança, revisamos casos reais de falhas (para aprender com eles) e exploramos tendências emergentes em segurança de contêineres e supply chain.

PRÁTICAS NAS PRINCIPAIS CLOUDS: GKE, EKS, AKS

Os provedores de Kubernetes gerenciado (Google GKE, Amazon EKS, Azure AKS) incorporaram **mecanismos nativos para fortalecer a segurança de workloads e segmentar permissões**.

O **Google Kubernetes Engine (GKE)** introduziu o conceito de **Workload Identity** para integrar pods com identidades do Google Cloud IAM. Em vez de usar chaves estáticas de contas de serviço GCP, o Workload Identity permite vincular uma ServiceAccount K8s a uma Conta de Serviço do GCP, usando OIDC sob o capô. Assim, um pod em GKE pode acessar APIs do Google Cloud (BigQuery, Cloud Storage etc.) **sem nunca possuir diretamente credenciais JSON**, mas sim através de tokens federados de curta duração.

Além disso, a Google recomenda fortemente **desativar o mecanismo legado de ABAC** (ainda disponível em configurações antigas) e usar somente RBAC aliado ao IAM do Google Cloud. O RBAC no GKE atua no nível do cluster e do namespace, enquanto o IAM atua no nível dos recursos de nuvem – ambos se complementam. Por exemplo, pode-se usar RBAC para restringir que apenas certa ServiceAccount possa usar um Secret específico, enquanto a Workload Identity assegura que aquela ServiceAccount, quando em um pod, terá somente permissão IAM designada (ex.: somente ler de um certo bucket).

Em linhas gerais, o GKE abraça *Princípio do Menor Privilégio* em múltiplas camadas: **RBAC para recursos do cluster e IAM para recursos cloud externos**, cada qual com políticas estritas e frequentemente condicionais (o IAM permite condições por horário, por atributos do recurso, etc., o que aumenta a precisão das políticas). Ainda, por padrão o GKE já isola contas de nó (cada nó com conta de

serviço do GCP limitada) e oferece criptografia de segredos com chave gerenciada por KMS.

O **Amazon EKS** em integrar Kubernetes com o ecossistema IAM. O destaque é o já citado **IRSA (IAM Roles for Service Accounts)**. Antes do IRSA, pods em EKS costumavam ter que usar as credenciais da IAM Role do nó (compartilhadas por todos pods naquele VM) ou inserção manual de keys. Com IRSA, cada ServiceAccount pode ter uma Role dedicada.

A AWS publica como *best practice* que **cada aplicação receba sua própria IAM Role** via IRSA, isolando acessos. Isso evita o problema de Roles de EC2 super permissivas serem abusadas por pods (um caso comum de ataque era comprometer um pod e usar as credenciais do nó – que às vezes vinham com permissão admin – para escalar no AWS). De fato, a recomendação é **não compartilhar** credentials e **não usar EC2 instance roles genéricas para pods**.

O EKS também integra com o IAM para controle de acesso ao próprio cluster: via o `aws-auth ConfigMap`, IAM usuários/grupos podem ser mapeados para permissões RBAC no cluster. Na prática, isso significa que a autenticação de usuários no Kubernetes pode ser delegada ao IAM (com MFA e todo o aparato), e o RBAC do cluster então reconhece, por exemplo, que o IAM grupo "DevOps" equivale ao grupo X dentro do cluster que tem certos roles.

Essa centralização pela IAM é importante para **auditoria unificada** – operações no cluster podem ser correlacionadas a identidades corporativas. Em termos de segregação, a Amazon incentiva o uso de **múltiplas contas AWS e múltiplos clusters** para isolar ambientes (produção vs staging, ou diferentes negócios). Cada cluster EKS tem seu próprio control plane isolado; quando há multi-tenancy dentro de um mesmo cluster, espera-se que RBAC e Namespaces façam o isolamento.

A AWS também fornece guias de segurança cobrindo *network policies*, encriptação etc., mas uma coisa a destacar: no modelo AWS, muitas vezes a segurança de cluster envolve mesclar **duas camadas de controle de acesso** (IAM e RBAC). Isso exige cuidado para não criar lacunas: por exemplo, um usuário IAM poderia ter acesso ao servidor da API, mas se o RBAC não der permissão a nada, ele efetivamente não consegue fazer mal. O inverso também vale: um ServiceAccount

pode ter binding RBAC amplo, mas se não tiver IRSA pra credencial nenhuma, seu impacto fora do cluster é nulo. O ideal é coordenar ambos.

Já o **Azure AKS**: O AKS se integra profundamente ao **Azure Active Directory (AAD)**. Por padrão, pode-se habilitar *Entra ID integration*, o que significa que usuários humanos que acessam o cluster devem autenticar via OAuth2 no AAD (em vez de certificados auto gerados). Essa abordagem centraliza identidades – quando alguém sai da empresa e é removido do Entra ID, automaticamente perde acesso ao cluster.

O Entra ID se torna fonte da verdade de usuários/grupos, e no cluster aplicam-se RoleBindings referenciando esses usuários (ex.: binding com kind: User, name: bob@empresa.com). Outra característica é o **Azure RBAC for Kubernetes**: a Microsoft permite opcionalmente gerenciar permissões Kubernetes via o Azure RBAC (que é diferente do RBAC interno do Kubernetes).

Isso pode simplificar atribuir papéis de leitura, escrita em namespaces diretamente via portal Azure, propagando para o cluster. Quanto a workloads, historicamente o AKS tinha o *AAD Pod Identity* (um componente que interceptava chamadas MSI e trocava tokens), mas foi substituído pelo **Azure Workload Identity** – que é similar ao IRSA/GKE Workload Identity, usando OIDC federado.

Com ele, uma ServiceAccount Kubernetes pode ser mapeada a uma **Managed Identity** do Azure, recebendo tokens de acesso Azure AD quando necessário. Assim, pods podem, por exemplo, pegar segredos no Azure Key Vault sem armazenar chave alguma, pois o Key Vault verá o token emitido pelo AAD para aquela identidade atrelada ao pod.

Em termos de segregação de permissão, a Azure enfatiza: use Namespaces para isolar equipes, use RBAC para restringir cada equipe ao seu NS, e possivelmente use **Azure Policy** (regras governamentais na subscription) para impor certas configurações (por exemplo, impedir que alguém desabilite network policy, ou exigir que volumes usem criptografia).

No nível de nós, o AKS por padrão executa todos pods de usuário em nós *agent* (pool separado do nó de sistema) e suporta Pools dedicados por workload se quiser garantir separação (um pool por tenant, usando *taints/tolerations* e *node affinity*). Em resumo, o AKS aproveita a forte identidade corporativa (Entra ID) e integra com permissões em duas frentes: acesso de usuários no cluster via Entra ID + RBAC, e

acesso de pods a Azure Services via Workload Identity. Estas características tornam o AKS o sistema mais seguro para Kubernetes entre os três maiores provedores.

Em todas as clouds, vemos um ponto comum: **movimento de credenciais estáticas para identidades federadas**. Isso melhora muito a segurança. Por exemplo, a falha conhecida como *SCARLETEEL* (2023) envolveu atacantes explorando um container vulnerável para roubar credenciais IAM de AWS e quase escalarem para roubar dados – a empresa alvo limitou danos justamente porque as credenciais IAM anexadas ao pod eram de escopo restrito.

Essa lição impulsiona práticas como IRSA: se Tesla tivesse usado IRSA com roles mínimas, talvez seu incidente de cryptojacking em 2018 (onde hackers acharam uma kube-dashboard aberta e usaram credenciais na cloud para minerar) teria menor impacto. Inclusive, no caso Tesla, relatórios apontam que o Dashboard sem senha estava rodando em um cluster cuja conta de nó tinha acesso amplo a S3, expondo dados sensíveis. Cloud providers então adicionaram medidas: hoje o EKS coloca restrições até nas políticas padrão de nós (ex.: o nó só pode acessar o API do EKS, não todos serviços).

Finalmente, vale mencionar **multi-tenancy versus multi-cluster**: algumas organizações optam por **vários clusters menores (descentralizado)** para separar workloads sensíveis, em vez de um cluster único multi-tenant (centralizado). Cada abordagem tem prós e contras. Multi-cluster oferece isolamento forte (limites de trust não são compartilhados), porém a sobrecarga de gestão multiplica.

Já multi-tenant (vários times no mesmo cluster) requer configurações rígidas de RBAC, Namespaces e políticas para evitar interferência. Existe consenso de boas práticas: se usuários não confiam muito uns nos outros (ex.: ambientes de clientes diferentes), use clusters distintos (*cluster per tenant*). Se são times internos com boa governança, multi-namespace pode ser aceito, mas então é obrigatório endurecer vários níveis de isolamento (rede, acesso a nós, scheduling, entre outros.).

Como referência, a estrutura **PEACH** da Wiz para multi-tenancy delimita 5 fatores de endurecimento (Privilege, Encryption, Access, Container, Hardware) e destaca que **RBAC no Kubernetes só é uma fronteira forte se bem configurado** (roles limitadas por namespace, sem permissões globais aos tenants, etc.). Os cloud

providers auxiliam nisso fornecendo *templates* e políticas prontas para configurar um cluster multi-tenant seguro, mas cabe aos administradores aplicá-las.

CASES REAIS DE VIOLAÇÕES E LIÇÕES APRENDIDAS

Em fevereiro de 2023, o ataque *SCARLETEEL* evidenciou o perigo de permissões excessivas. Um pod comprometido foi capaz de obter credenciais temporárias da AWS através de uma Role anexa e enumerar diversos recursos cloud. Felizmente, a role roubada tinha escopo limitado, impedindo que os atacantes acessassem tudo – ainda assim eles conseguiram extrair alguns dados de propriedade intelectual.

A lição aqui é dupla: (a) **nunca dar permissões de nuvem além do necessário a pods** – no caso, a Role deveria talvez nem permitir listar instâncias EC2, pois os atacantes usaram isso para descobrir alvos; (b) mesmo aplicações internas podem ser porta de entrada, então aplicar princípio de menor privilégio no IAM limitou o estrago (o artigo nota que a empresa “limitou corretamente o escopo da identidade roubada, o que atenuou o ataque”). Em outras palavras, se aquela ServiceAccount tivesse role admin na cloud, as consequências seriam desastrosas (exfiltração total de dados). **A configuração correta do IRSA salvou o dia.**

Já em 2018, pesquisadores da RedLock encontraram um dashboard do Kubernetes da Tesla **sem autenticação**, acessível publicamente na internet. Através dele, obtiveram acesso ao cluster e descobriram credenciais de acesso armazenadas em um pod (de um serviço de monitoramento) que possuíam permissões amplas na infraestrutura AWS da Tesla.

Os invasores então rodaram contêineres para minerar criptomoeda e potencialmente poderiam ter roubado dados sensíveis armazenados no S3. O impacto só não foi pior porque foi detectado e interrompido. Lições: nunca expor interfaces de administração sem proteção – hoje, é consenso não rodar o *Kubernetes Dashboard* em produção ou, se rodar, bloqueá-lo via firewall/VPN e removê-lo de roles privilegiadas.

Além disso, **nunca incluir segredos estáticos em pods** (na época, aparentemente havia credenciais AWS em plaintext). Atualmente, coloca-se segredos

no Kubernetes Secrets com encriptação, e idealmente usam-se credenciais dinâmicas. Esse case ilustra a importância de **Secure Defaults**: O dashboard do Kubernetes clássico rodava com uma ServiceAccount que muitas vezes tinha mais acesso do que deveria.

A recomendação pós-incidente foi rodar dashboards isolados e sem permissão de alterar recursos, somente visualizar, e de preferência em clusters de teste. Em suma, seguiu-se a máxima “Não exponha o kube-dashboard; se precisar, **não lhe dê conta de serviço privilegiada** e proteja seu endpoint”.

Houve incidentes em que atacantes sem acesso de admin aproveitaram permissões leitura/escrita indevidas em ConfigMaps para injetar configurações maliciosas (por ex, alterando configuração do CoreDNS via ConfigMap se tivesse permissão). Isso reforça: **até permissões aparentemente inocentes devem ser tratadas com cuidado**. RBAC deve impedir que um componente comprometa outro. Em ambientes multi-tenant, é recomendável ativar audit logs e monitorar ativamente uso de credenciais. Por exemplo, a Falco (projeto CNCF) e o Kyverno podem detectar se um pod está usando um token ServiceAccount para ações suspeitas.

Em um teste de penetração, uma equipe demonstrou que uma ServiceAccount com capacidade de criar pods no cluster poderia ser usada para agendar um pod privilegiado montando o socket do Docker do host, efetivamente escapando para o nó e comprometendo todo cluster (já que do nó poderiam ler tokens de outras SAs).

Após esse reporte, a empresa adotou *Pod Security Policies* (na época) e agora *Pod Security Standards/Admission* para garantir que mesmo que alguém tenha permissão de criar pods, não consiga criar contêiner privilegiado ou hostPath indiscriminadamente. Moral: mesmo dentro do escopo “normal” do RBAC, certos verbs podem ser perigosos (ex.: permitir criação de pods sem restrições a usuários não confiáveis é arriscado). Deve-se complementar RBAC com políticas de pod (como as do OPA/Gatekeeper) para reforçar limites.

No geral, esses cases mostram que **erros de configuração são a maior causa de violações** – seja um serviço exposto, uma credencial não rotacionada, uma ServiceAccount com poderes demais, ou imagens não verificadas. A partir deles, o mercado reagiu com várias tendências que detalhamos a seguir.

RBAC CENTRALIZADO VS DESCENTRALIZADO – TOPOLOGIAS VISUAIS

Quando falamos em **RBAC centralizado**, tipicamente nos referimos a um modelo em que *uma entidade administrativa controla todas as definições de roles e bindings do cluster*. Isso acontece, por exemplo, quando há um time de plataforma que define papéis padrão (leitor, editor, admin) e os aplica a todos projetos, evitando que desenvolvedores individuais criem roles ad-hoc. Essa centralização facilita auditoria (tudo num só lugar) e consistência (padrões uniformes), porém pode ser menos flexível e tornar o time central um gargalo.

Por outro lado, um RBAC **descentralizado** permitiria que cada time/projeto definisse suas Roles dentro de seu namespace. Kubernetes permite isso até certo ponto – desenvolvedores com acesso a um namespace podem criar Roles naquele namespace e RoleBindings associando suas ServiceAccounts. O cluster-admin ainda controla quem pode criar roles (via RBAC do próprio RBAC!), mas em clusters maiores vemos frequentemente delegação: *admin central dá permissão ao líder do time X para gerenciar RBAC dentro do namespace X*. Visualmente, podemos imaginar dois cenários:

- **Centralizado:** um único grafo RBAC englobando todo cluster. As roles (particularmente ClusterRoles) são definidas globalmente. Exemplo: roles “Dev-Reader”, “Dev-Editor” existem cluster-wide e são vinculadas a usuários de cada time conforme necessidade. Todos bindings referenciam essas mesmas roles padrão. Administradores globais revisam mudanças de roles.
- **Descentralizado:** múltiplos grafos RBAC segmentados por namespace (com possíveis overlaps). Cada namespace *N* tem suas Roles *RN1*, *RN2*, etc., e bindings respectivos. Não há (ou há poucas) ClusterRoles genéricas – talvez cada time defina até suas ClusterRoles se precisar acesso cross-namespace. A revisão de permissões acontece localmente por namespace.

Centralizado tende a ocorrer em ambientes regulados, onde se quer garantir que ninguém tenha a *mais* sem aprovação formal. Por exemplo, instituições financeiras podem ter um *Change Advisory Board* que aprove criação de qualquer Role nova. Já descentralizado aparece em empresas tech ágeis onde se confia nos times para se auto-gerirem, priorizando agilidade.

A consequência é que a postura de segurança depende muito da maturidade: um time descuidado poderia, sem fiscalização, criar RoleBindings dando acesso amplo e comprometer isolamentos. Ferramentas como o **vCluster** e outras soluções multi-tenant às vezes criam uma *ilusão de descentralização*, dando a cada tenant uma visão de RBAC isolada (um *virtual cluster* por namespace).

Na comparação de topologias, com RBAC centralizado a **política é única**, fácil de verificar globalmente, mas potencialmente conflitante com necessidades específicas (ex.: talvez uma equipe precise de permissão incomum, e o admin central demora para adicionar na Role global). Com RBAC descentralizado, a **política é fragmentada**, e cada equipe faz o seu, adequando perfeitamente ao caso, mas para auditar fica difícil (teria que somar todos roles de todos namespaces mentalmente). Uma abordagem híbrida é comum: políticas centrais definem limites (ex.: “nenhum usuário comum terá ClusterAdmin”), mas dentro do sandbox cada um se ajeita. Isso é semelhante a um sistema federalista.

Em ambos os casos, convém acompanhar com **Network Policies** e outros mecanismos, pois RBAC só cuida do API Server. Topologia descentralizada exige rede restrita entre tenants (para que um pod de um time não acesse serviço de outro). Topologia centralizada muitas vezes coincide com maior confiança interna e rede mais aberta – o que exige mais diligência ainda no RBAC. Ou seja, a escolha impacta outras camadas: no cluster único multi-tenant (central), **NetworkPolicy** e **PodSecurity** são obrigatórios; no multi-cluster (descentral total), isolar rede é trivial (clusters separados) e políticas de pod podem até ser relaxadas por cluster, já que cada um serve um único stakeholder.

Não há bala de prata, e por isso muitas empresas primeiramente isolam tenants críticos em clusters separados (descentralização de infraestrutura) e dentro de um cluster compartimentam por namespace com RBAC (centralização parcial de controle). Essa abordagem mista visa colher benefícios de ambos os lados: clusters separados para grandes domínios (prod, dev, clientes diferentes) e RBAC consistente dentro de cada cluster para segmentar micro-domínios (equipes, aplicações).

Tendências emergentes: SBOM, assinaturas de imagem, supply-chain security

A segurança de software containerizado avançou além do cluster em si – engloba toda a **cadeia de suprimentos de software (supply chain)**. Três tendências se destacam atualmente:

SBOM é um manifesto listando **todos os componentes** de um software (bibliotecas, pacotes, versões, hashes). A ideia é trazer transparência semelhante à indústria manufatureira: você sabe exatamente quais “peças” entraram no seu aplicativo/container. Em 2021-2022, reguladores como o governo dos EUA (via ordem executiva) e padrões do NIST começaram a exigir SBOMs para software crítico.

Em 2025, isso se tornou uma prática comum – ferramentas como Syft, Anchore, CycloneDX geram SBOMs de contêineres e aplicações. No contexto Kubernetes, surgiu até o conceito de **KBOM (Kubernetes BOM)**, que expande o SBOM para enumerar também componentes do orquestrador (versão do kubelet, do apiserver, imagens de sidecars, CRDs instaladas etc.).

Qual o benefício de SBOM? Gestão de vulnerabilidades e compliance.

Com um SBOM anexado à sua imagem, quando sai uma CVE (falha) numa biblioteca X versão Y, fica trivial (via ferramentas automatizadas) localizar quais imagens contém aquele componente. Sem SBOMs, empresas gastam muito tempo e scans tentando descobrir se estão afetadas. Com SBOM, é consulta direta numa lista.

Além disso, SBOMs ajudam a cumprir requisitos de licenciamento (saber licenças de cada pacote) e evitar componentes proibidos. Em Kubernetes, o SBOM pode ser verificado em pipelines CI e até em admissão: por exemplo, impedir implantações de imagens que contenham bibliotecas fora de conformidade. Esse movimento reflete um amadurecimento: **a segurança passa a olhar dentro da imagem**, não só para configurações exteriores. Vale notar que a CNCF e outras oferecem padrões (SPDX, CycloneDX) para SBOMs, e já existem recomendações de *guardar SBOMs em repositórios junto com as imagens*.

Uma grande preocupação é garantir a integridade e origem das imagens de contêiner. Ferramentas como **Cosign (do projeto Sigstore)** possibilitam assinar criptograficamente imagens container e armazenar as assinaturas na própria registry (geralmente como artefatos associados à imagem).

A ideia é semelhante a assinar um pacote .deb ou RPM. No Kubernetes, podemos configurar uma política para **só executar imagens assinadas por entidades confiáveis**. Por exemplo, configurar um *admission controller* (como o **policy-controller do Sigstore** ou o **Connaisseur**) que, ao ver um Pod ou Deployment, verifica a assinatura da imagem referenciada antes de permitir.

Caso a imagem não possua assinatura válida, o deploy é bloqueado. Isso mitiga ataques de supply chain onde alguém injeta uma imagem maliciosa no registro ou manipula tags. Com assinaturas, mesmo que um invasor publique um "nginx:latest" falso, a cluster não o aceitará se não estiver assinado pela chave que confiamos (por exemplo, a chave da equipe DevOps).

Em 2025, esse tipo de verificação está ficando mais comum e até sendo incorporado nativamente: o Kubernetes 1.30 trouxe um recurso de *ValidatingAdmissionPolicy* que pode ser usado para checar anotações de assinatura. Também o próprio processo de release do Kubernetes agora assina seus artefatos com Cosign, reforçando a importância.

Em termos de tendência, **Keyless signing** (assinatura sem gerenciar chaves manualmente, usando infra de transparência como Rekor e fulcio) torna mais fácil adotar. Empresas estão integrando assinatura no CI: toda imagem sai da pipeline já assinada automaticamente. E não apenas imagens: manifests YAML, pacotes Helm, etc., também podem ser assinados (o Helm já suporta assinatura de charts há tempos).

Além de SBOM e assinatura, há iniciativas amplas como **SLSA (Supply Chain Levels for Software Artifacts)**, que define níveis de garantia no pipeline de software. O objetivo é evitar casos como o ataque *SolarWinds* ou compromissos de bibliotecas upstream. Para Kubernetes, supply chain security significa garantir que **o que você roda no cluster é exatamente o que deveria rodar**.

Isso engloba ter pipelines CI/CD seguras (isoladas, com acesso mínimo), usar bases de código confiáveis e verificar integridade de dependências (*dependency poisoning* é uma ameaça real, vide pacotes typosquatting no npm/pip), implementar política de proveniência, por exemplo, somente imagens construídas via nosso CI e assinadas serão implantadas (não permitir alguém rodar imagem de docker hub público sem validação) e monitorar ambientes de build tanto quanto os de produção,

pois se um invasor injeta backdoor na fase de build, vai parar em produção de forma invisível.

Dentro do cluster, surgiram componentes de segurança focados nisso, como o **Kyverno** e **OPA Gatekeeper** que podem validar se um Deployment refere imagens de registries aprovadas, se a imagem tem um ID imutável (digest) em vez de tag mutável, se foi escaneada por vulnerabilidades, entre outros casos.

E falando em **escaneamento**, ferramentas de *container scanning* estão se integrando à admissão: pode-se bloquear imagens que tenham CVEs críticas não corrigidas, por exemplo, usando políticas. Essa é outra tendência: não basta confiar que time corrigiu: o cluster pode **impor política de CVE** (ex: "Não permita rodar imagem com CVSS>9 aberto"). Claro que isso tem complexidade (falso positivo, timing de patches), mas demonstra a mentalidade proativa.

Outra frente é a **identidade de desenvolvedor e commit**, com iniciativas como **Sigstore Cosign & Git commit signatures** e **Enforced Code Review** se relacionam: garantir que código em main foi revisado e assinado, garantir que build puxou aquele código puro, e cluster roda build correspondente – fechando o laço de confiança.

É um problema complexo, mas há progressos: o Tekton Chains, por exemplo, pode gerar atestados que vinculam imagem -> repositório git commit, e essas atestações podem ser verificadas no deploy (chamado de **Verified Image Provenance**). A CNCF também encoraja uso de repositórios declarativos (GitOps) com commit signing e verificações no CD (Flux, ArgoCD suportam validar GPG signatures dos manifests). Tudo isso compõe a **segurança da cadeia de suprimentos de software**.

A proteção deixou de ser apenas "impeça intruso de entrar" e virou "presuma que intruso pode entrar, então minimize o impacto e dificulte movimentos". Fornecer identidades fortes (JWT, X.509), políticas expressivas bem analisadas, rotação constante de credenciais, verificar o que roda (assinaturas) e saber do que é feito (SBOM) – todos são elementos de uma postura *defense-in-depth* adequada aos desafios atuais.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, revisamos diversos aspectos avançados de segurança em clusters Kubernetes. Vimos como workloads devem usar ServiceAccounts próprias para garantir identidade isolada e seguir o princípio do menor privilégio. Aplicamos RBAC na prática, criando Roles e RoleBindings mínimas e comparando com o modelo ABAC, destacando a simplicidade e auditabilidade do RBAC.

Exploramos a integração com identidades externas em nuvens públicas, como IRSA (EKS) e Workload Identity (GKE/Azure), usando tokens OIDC para acesso seguro a recursos externos sem credenciais fixas. Aprendemos a automatizar certificados TLS com cert-manager, construindo um cenário de PKI interna com Issuers e Certificates gerenciados automaticamente.

Nos fundamentos teóricos, consolidamos a compreensão de RBAC vs. ABAC e revimos conceitos de PKI e X.509, entendendo como o Kubernetes compartimenta autoridades certificadoras para conter comprometimentos. Discutimos política como código com OPA/Rego, destacando a expressividade e os desafios de verificabilidade. Abordamos a rotação de segredos e gerenciamento de chaves, mostrando como fazer isso de forma coordenada para evitar falhas, usando múltiplas chaves no config de criptografia do Kubernetes.

Analizamos ainda casos reais de incidentes de segurança, como o da Tesla e o ataque SCARLETEEL, aprendendo lições práticas sobre os riscos de configurações incorretas e a eficácia de boas práticas. Por fim, conectamos tudo com a segurança da cadeia de suprimentos, entendendo o papel de SBOMs, assinatura de imagens, validação no deploy e frameworks como SLSA. A aula cobriu desde operações diárias com identidades e roles até fundamentos formais por trás de políticas e certificados, preparando para práticas modernas de DevSecOps em ambientes Kubernetes seguros e confiáveis.

Esta aula cobriu do **básico ao avançado**: desde configurar identidades e roles no dia a dia do cluster, até compreender e aplicar princípios matemáticos e lógicos por trás das políticas e certificados, e se antecipar às evoluções que o mercado e a comunidade estão adotando para tornar ambientes Kubernetes cada vez mais seguros e confiáveis. **Bons deploys!**

REFERÊNCIAS

ALPHABRAVO ENGINEERING. **Building Trust – Establishing Your Kubernetes Root CA**. Alphabravo Blog. 2025. Disponível em: <https://alphabravo.io>. Acesso em: 7 set. 2025.

BURILLO, M. **Kubernetes RBAC and TLS certificates – Kubernetes security guide (part 1)**. Sysdig Blog. 2018. Disponível em: <https://sysdig.com>. Acesso em: 7 set. 2025.

EHRMAN, N. **7 GKE Security Best Practices**. Wiz Blog. 2024. Disponível em: <https://www.wiz.io>. Acesso em: 7 set. 2025.

GOROVOY, I. **Internal TLS communication between pods in Kubernetes using cert-manager**. Medium. 2025. Disponível em: <https://medium.com>. Acesso em: 7 set. 2025.

HASHICORP. **Rotated vs. dynamic secrets: Which should you use?** HashiCorp Blog. 2024. Disponível em: <https://www.hashicorp.com>. Acesso em: 7 set. 2025.

LEMONS, R. **Pernicious Permissions: How Kubernetes Cryptomining Became an AWS Cloud Data Heist**. Dark Reading. 2023. Disponível em: <https://www.darkreading.com>. Acesso em: 7 set. 2025.

LUKŠA, M. **Kubernetes in Action**. Shelter Island: Manning Publications. 2018. ISBN 978-1-61729-372-6.

MICROSOFT. **Best practices for authentication and authorization in Azure Kubernetes Service (AKS)**. Microsoft Learn. 2024. Disponível em: <https://learn.microsoft.com>. Acesso em: 7 set. 2025.

NAIK, A. **Hacking and Securing Kubernetes: A Deep Dive into Cluster Security**. InfoSec Write-ups on Medium. 2025. Disponível em: <https://medium.com>. Acesso em: 7 set. 2025.

SYSDIG. **How to secure Kubernetes deployment with signature verification**. Sysdig Blog. 2022. Disponível em: <https://sysdig.com>. Acesso em: 7 set. 2025.

UPWIND. **EKS Security Best Practices**. Upwind by Aqua Security. 2023. Disponível em: <https://www.upwind.io>. Acesso em: 7 set. 2025.

WALKER, J. **Control your role! Kubernetes RBAC explored**. Snyk Blog. 2023. Disponível em: <https://snyk.io>. Acesso em: 7 set. 2025.

WIZ RESEARCH. **What is a Kubernetes Bill Of Materials (KBOM)?** Wiz Academy 2025. Disponível em: <https://www.wiz.io>. Acesso em: 7 set. 2025.

PALAVRAS-CHAVE

Kubernetes. Segurança. RBAC. ABAC.

EMSE



POS TECH